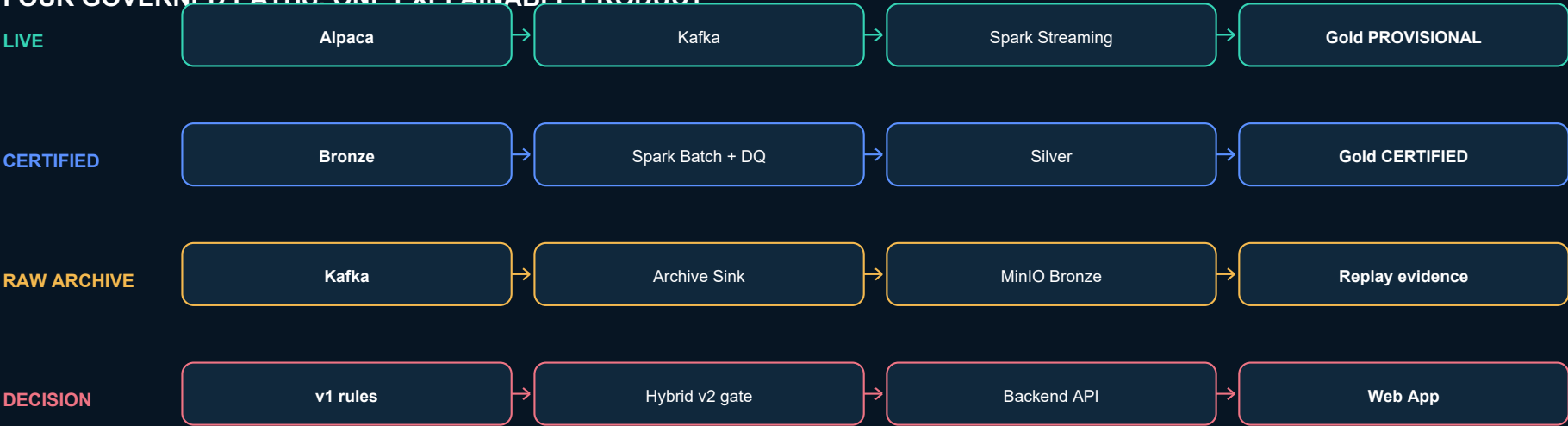


Technical Architecture

A local, reproducible and governed market-data platform

FOUR GOVERNED PATHS, ONE EXPLAINABLE PRODUCT



VERSION	2.0
VERIFIED	2026-09-17
STATUS	Final project baseline

Document status: final project baseline

Architecture version: 2.0

Last verified: 2026-09-17 Scope: local Docker Compose platform for market data, SEC fundamentals, governed analytics, backtesting, and explainable decision support

1. Executive summary

MarketPilot is a local, reproducible data platform that turns live and historical US-equity data into an auditable research product. It follows one central rule: speed must never erase evidence. Live data is useful immediately as **PROVISIONAL**; bounded post-market processing rebuilds the same session from immutable source data, applies data-quality gates, and publishes **CERTIFIED** results.

Docker Compose owns every long-running service. Airflow schedules and monitors only bounded work. Kafka provides the event log, Spark performs streaming and batch computation, MinIO stores immutable and analytical files, MariaDB serves application-ready Gold data, and the browser communicates only with the Backend API.

The application supports research and decision support. It does not place orders, connect to brokerage balances, promise returns, or replace human judgment.

2. Product and quality goals

The platform is designed to answer five questions:

- 1 What is happening in the market now?
- 2 Can the user trust the displayed data?
- 3 What company information was known at that moment?
- 4 What risk, invalidation point, and possible reward define the scenario?
- 5 Can the result be reproduced from source, code version, run, and data version?

The architecture therefore prioritizes immutable raw evidence, explicit schema versions, UTC event time, idempotent writes, quality gates, point-in-time features, and visible **PROVISIONAL**, **CERTIFIED**, **PREVIEW**, and **FALLBACK** states.

UTC remains the storage and API contract. The Web App converts timestamps only at the presentation boundary with the named **Asia/Jerusalem** time zone, labels them as Israel time, and therefore follows Israeli daylight-saving changes without altering event identity or market-session logic.

3. System context

Actor or system	Interaction with MarketPilot
Alpaca Market Data	Live WebSocket bars, bounded historical bars, and corporate actions
SEC EDGAR	Filing metadata and Company Facts/XBRL fundamentals
Research user	Uses dashboards, watchlists, portfolio assumptions, opportunities, and backtests
Docker Compose	Supervises the complete local runtime
Airflow	Schedules and monitors bounded acquisition, transformation, quality, archive, backtest, and model-training jobs

Tracked universe: [AAPL](#), [MSFT](#), [AMZN](#), [NVDA](#), [GOOGL](#), [META](#), [TSLA](#), [JPM](#), [UNH](#), [XOM](#), and [SPY](#). [SPY](#) is both a visible asset and the market benchmark.

4. Governing data paths

4.1 Live path

```
Alpaca -> market-producer -> Kafka -> Spark Structured Streaming
      -> MariaDB Gold PROVISIONAL -> Backend API -> Web App
```

The producer normalizes market bars to [MarketBarV1](#) and publishes keyed Kafka events. Spark Structured Streaming validates and upserts near-real-time Gold rows. A durable checkpoint allows restart without inventing a second business record.

4.2 Raw archive path

```
Kafka -> raw-archive-sink -> MinIO Bronze -> offset commit
```

The archive consumer writes the exact event to immutable Bronze storage before committing its Kafka offset. Topic, partition, and offset remain part of the object path and lineage. This path is independent from the live serving path.

4.3 Certified batch path

```
MinIO Bronze -> Spark Batch -> MinIO Silver -> Data Quality
      -> Spark Batch -> MariaDB Gold CERTIFIED
```

Airflow submits bounded Spark applications through `SparkSubmitOperator`. A failed completeness, consistency, or freshness gate blocks certified publication. The daily path never starts or stops the streaming service.

4.4 Historical acquisition path

```
Airflow -> Alpaca IEX REST -> content-addressed source pages
      -> Kafka backfill topic -> Bronze offset barrier
      -> Silver -> DQ -> Gold CERTIFIED -> Backtest
```

Historical data uses the same contracts and certification gates as live data. A dedicated Kafka topic prevents backfill traffic from entering the live streaming job. Requests are limited to 31-day windows and are idempotent.

4.5 SEC and corporate context

```
Airflow -> SEC EDGAR / Alpaca corporate actions
      -> immutable Bronze JSON -> normalized Gold facts
      -> fundamental and split-adjusted analytical features
```

SEC requests use a declared `User-Agent`, rate limiting, retries, and accession-number deduplication. Company Facts are point-in-time filtered so a model snapshot cannot see a filing published in the future. Raw price bars are never rewritten for splits; analytical copies are adjusted.

5. Runtime ownership

Lifecycle owner	Components	Rule
Docker Compose	Kafka, producer, archive sink, Spark master/worker/streaming, MinIO, MariaDB, Airflow services, API, Web, monitor, UI helpers	Long-running processes and infrastructure
Airflow	SEC polling, corporate actions, daily close, historical acquisition, replay, backtest, compaction, archive, hybrid training	Bounded work with a defined start and end
Spark	Streaming and batch computation	Computes data; does not own business scheduling

Airflow never controls the lifecycle of Kafka, Spark Streaming, MariaDB, MinIO, the Backend API, or the Web App. This boundary is enforced by ADR-002 and ADR-003.

6. Implemented Docker Compose topology

The project defines 19 Compose services, including two finite initialization services.

Group	Services	Purpose
Data plane	<code>kafka</code> , <code>market-producer</code> , <code>raw-archive-sink</code> , <code>spark-master</code> , <code>spark-worker</code> , <code>spark-streaming</code> , <code>minio</code> , <code>mariadb</code>	Ingestion, transport, compute, and storage
Control plane	<code>airflow-db</code> , <code>airflow-init</code> , <code>airflow-scheduler</code> , <code>airflow-api-server</code> , <code>airflow-dag-processor</code>	Workflow metadata and bounded orchestration
Serving plane	<code>backend-api</code> , <code>web-app</code>	Product API and browser experience
Operations	<code>operational-monitor</code> , <code>platform-init</code>	Read-only probes and idempotent platform bootstrap
Local inspection	<code>kafka-ui</code> , <code>spark-ui-proxy</code> , <code>adminer</code>	Demonstration and developer inspection only

Every technically suitable long-running service has a healthcheck and restart policy. Readiness-sensitive dependencies use health-based startup conditions.

7. Network and trust boundaries

Docker network	Exposure	Members and purpose
<code>data-plane</code>	Internal	Kafka, Spark, storage, ingestion, archive, monitor
<code>control-plane</code>	Internal	Airflow metadata and control services
<code>serving-plane</code>	Host-facing where required	Web, API, Airflow UI, MinIO Console, Kafka UI, Adminer, Spark UI proxy

The browser has no MariaDB or MinIO credentials. It calls relative `/api/` routes through Nginx, which proxies to the Backend API. The API uses parameterized SQL, bounded ranges, response models, symbol validation, and least-privilege database identities.

Host-facing demonstration endpoints bind to loopback where applicable. The local project is not a production security boundary; production deployment would additionally require authentication, TLS, secret management, network policy, and audited user identity.

8. Persistent state

Named volumes protect Kafka logs, MinIO objects, MariaDB data, Airflow metadata/logs/authentication files, and Spark checkpoints. The important volumes are:

- `kafka-data`
- `minio-data`
- `mariadb-data`
- `airflow-db-data`
- `airflow-logs`
- `airflow-auth`
- `spark-checkpoints`

MinIO buckets separate Bronze, Silver, streaming checkpoints, analytics/model artifacts, and long-term archives. Restart tests verify that Spark resumes from checkpoints and business-key upserts prevent duplicate rows.

9. Kafka and event contracts

Topic	Key	Consumer intent
<code>market.bars.1m.v1</code>	Symbol	Live streaming and raw archive
<code>market.bars.1m.backfill.v1</code>	Symbol	Historical certification and raw archive; excluded from live streaming
<code>market.bars.1m.dlq.v1</code>	Event identifier	Quarantined malformed or rejected events

`MarketBarV1` carries a deterministic event ID, symbol, UTC event time, interval, OHLCV values, source, schema version, and ingestion timestamp. The business key is symbol plus interval plus event timestamp. Invalid OHLC relationships, non-positive prices, negative volume, or unsupported schema versions are quarantined rather than silently dropped.

10. Medallion storage model

Bronze

Immutable, append-oriented evidence in MinIO. Market event paths include source, event, date, symbol, topic, partition, and offset. SEC and corporate-action pages are stored content-addressed with hashes.

Silver

Canonical Parquet with typed fields, UTC timestamps, deduplication, normalized schemas, validation flags, and controlled partition layout. Weekly compaction reduces small files without changing logical rows.

Gold

MariaDB holds the serving model: market bars, indicators, signals, filings, fundamentals, corporate actions, opportunities, evaluations, user research state, backtest summaries, quality results, watermarks, archives, and hybrid-model metadata. Raw payloads and full-resolution analytical artifacts remain in object storage.

11. Gold data domains

Domain	Representative tables
Market and lineage	<code>dim_symbol</code> , <code>fact_market_bar_1m</code> , <code>etl_watermark</code> , <code>data_quality_result</code>
Analytics	<code>fact_indicator_1m</code> , <code>fact_signal</code>
SEC and fundamentals	<code>fact_sec_filing</code> , <code>fact_fundamental_metric</code> , <code>fact_corporate_action</code>
Decision support	<code>fact_opportunity_recommendation</code> , <code>fact_recommendation_evaluation</code> , <code>fact_decision_alert</code> , <code>shadow_mode_status</code>
User research state	<code>user_portfolio</code> , <code>user_position</code> , <code>user_watchlist_symbol</code>
Backtesting	<code>dim_strategy</code> , <code>fact_backtest_run</code> , <code>fact_backtest_result</code> , <code>fact_backtest_equity_daily</code>
Hybrid model	<code>fact_decision_feature_snapshot</code> , <code>fact_decision_label</code> , <code>decision_model_registry</code> , <code>fact_decision_model_prediction</code>
Operations and recovery	<code>archive_manifest</code> , <code>archive_restore_result</code>

All mutating paths use deterministic keys and idempotent writes. Publication state and lineage distinguish live, certified, historical, model-preview, and fallback outputs.

12. Airflow DAG catalogue

DAG	Trigger	Bounded responsibility
<code>sec_polling</code>	Scheduled	Discover, download, deduplicate, and publish filing/company facts
<code>corporate_actions</code>	Scheduled/manual	Archive and normalize splits and dividends
<code>daily_market_close</code>	Weekdays after market close in <code>America/New_York</code>	Exchange-calendar gate, Bronze-to-Silver, DQ, Certified Gold, analytics, evaluation
<code>historical_market_backfill</code>	Manual, maximum 31 calendar days	Historical acquisition through Kafka, Bronze, certification, and backtest
<code>backfill_replay</code>	Manual	Reprocess selected Bronze scope idempotently
<code>historical_backtest</code>	Manual	Reproducible strategy evaluation on Certified Gold
<code>weekly_compaction</code>	Weekly	Compact Silver Parquet and verify logical equivalence
<code>annual_archive</code>	Annual/manual	Export, hash, manifest, and restore-test closed data
<code>hybrid_model_training</code>	Manual	Dataset, labels, chronological validation, artifact registration, and fail-closed promotion gates

Partition-mutating DAGs use `max_active_runs=1`. Shared external and compute capacity is serialized through `sec_api_pool`, `alpaca_api_pool`, and `spark_batch_pool`.

13. Data quality and publication

Quality checks cover freshness, completeness, duplicates, nulls, OHLC consistency, expected exchange-session bars, schema versions, and lineage. Historical IEX certification uses the feed-aware policy in ADR-010: every requested symbol must be present with at least 35% of expected regular-session minutes, while the complete universe must reach at least 80% aggregate coverage. Missing minutes are never synthesized. The daily close pipeline is intentionally stricter and may fail when a partial IEX feed misses expected bars.

Failure is visible and recoverable. A failed daily run is retained as audit evidence; a bounded historical repair can reacquire the same date through the governed path and publish a certified replacement without erasing the failed run.

14. Analytics and backtesting

Certified bars feed versioned indicator jobs for [SMA](#), [EMA](#), [RSI](#), [MACD](#), [ATR](#), realized volatility, and volume ratios. Multi-timeframe decision features use 5-minute, 15-minute, hourly, and daily windows; the one-minute series is primarily for display and fast context.

Backtests read only Certified Gold. A signal produced from bar [t](#) is applied to a later bar, preventing look-ahead. Costs and slippage reduce returns. Results include immutable run parameters, code/data versions, benchmark comparison to [SPY](#), a bounded equity curve in MariaDB, and full-resolution Parquet in MinIO.

15. Decision Intelligence v1

The deterministic rules remain authoritative. They calculate:

- [Buy Zone](#) from support, EMA context, and ATR;
- [Stop](#) from technical invalidation and volatility;
- [Target 1](#) and [Target 2](#) from resistance and risk multiples;
- technical, fundamental, and combined scores;
- risk/reward and position sizing under portfolio limits;
- actions: [BUY_ZONE](#), [WAIT](#), [WATCH_BREAKOUT](#), [AVOID](#), or [INSUFFICIENT_DATA](#).

The first 20 successful live certified sessions are Shadow Mode. Historical backfills improve evidence but never advance this safety gate. Software may reach [READY_FOR_REVIEW](#); a human must approve any later activation.

16. Hybrid Decision Intelligence v2

Phase 15 adds a probability layer without replacing deterministic price levels or risk gates. The target is the conditional probability that [Target 1](#) is reached before [Stop](#) within ten exchange sessions, after the market actually entered the published Buy Zone.

The training design compares calibrated Logistic Regression and Histogram Gradient Boosting using expanding chronological walk-forward folds. [NO_ENTRY](#) is measured separately and is not mislabeled as a losing trade. Point-in-time features prevent future market or SEC information from leaking into the past.

Promotion requires 24 months of suitable history, at least 300 valid entries, at least 50 successes, coverage across eight assets and four quarters, acceptable Brier score, PR-AUC, calibration, positive expected R after friction, diversification, and no failed quality window. The model also requires 20 new v2 live certified sessions and explicit human approval.

Current final-project state is **FALLBACK**: no trained model has been registered because the evidence gate is not yet satisfied. v1 rules continue to control the product. The UI deliberately shows **Rule Score**, **Model Probability**, **Data Confidence**, **Expected R**, and model state as separate concepts; it never invents a probability.

17. Failure, restart, and recovery

Failure	Expected behavior
Producer or WebSocket disconnect	Reconnect with bounded exponential backoff; no fabricated bars
Kafka/archive failure	Offset is not committed before Bronze persistence
Spark Streaming restart	Resume from durable checkpoint and idempotent business keys
Batch/DQ failure	Certified publication and watermark advancement are blocked
Historical retry	Content hashes, event IDs, and upserts prevent duplicated business data
Missing or corrupt model artifact	Switch to FALLBACK ; retain v1 rules; never serve a stale probability
Archive restore drill	Verify inventory and checksums, then restore only to an isolated schema

The operational monitor performs read-only dependency and freshness probes and emits state changes instead of repeated alerts.

18. Security and secrets

Secrets live in an untracked `.env`; `.env.example` contains safe placeholders. Browser code contains no infrastructure credentials. Database identities are separated by responsibility: ingestion, certification, SEC publication, read-only application access, and bounded user-state writes.

The local stack is intentionally scoped to loopback development and demonstration. Production hardening would add managed secret rotation, user authentication, role-based authorization, TLS, network policy, audit logging, and a managed object store/database.

19. Current verified evidence

The release snapshot verified on 2026-09-17 contains:

- 11 tracked symbols;
- 162,743 Gold one-minute bars, including 154,206 **CERTIFIED** rows;
- 937 SEC filing records;
- 76 certified historical exchange sessions from 2024-09-17 through 2026-09-16;
- 12 published backtest runs;
- v1 Shadow Mode at 2 of 20 live certified sessions;
- v2 hybrid model in explicit **FALLBACK**, with no fabricated probability.

These values are evidence snapshots, not hard-coded product claims. The application and demo preflight read current status from the API.

20. Capacity, limitations, and roadmap

The complete local stack is resource-heavy. A practical workstation target is eight CPU cores, 16 GB RAM, and SSD storage. Spark must leave enough memory for Kafka, Airflow, MariaDB, MinIO, the API, and the operating system.

Known limitations are explicit: the free IEX feed is partial, the dataset is not yet 24 months, dividends are archived but not yet credited in price-return backtests, the fixed universe introduces survivorship bias, and local Compose is not a production availability or security design.

The next justified steps are additional governed history, v2 training and validation, completion of the live Shadow Mode gate, human review, and only then broader data coverage or cloud deployment. Elastic observability, SIP data, S3, and production authentication remain optional future work, not hidden dependencies of the final project.

21. Architecture decision records

The accepted ADRs are the binding rationale for storage, orchestration, lifecycle, publication, backtesting, historical acquisition, decision intelligence, and hybrid modeling:

- ADR-001 — storage strategy
- ADR-002 — Airflow boundary

- ADR-003 — streaming lifecycle
- ADR-004 — provisional and certified publication
- ADR-005 — historical backtesting
- ADR-006 — optional local observability
- ADR-007 — certified historical acquisition
- ADR-008 — decision intelligence
- ADR-009 — hybrid decision intelligence
- ADR-010 — feed-aware IEX historical coverage

The Mermaid sources under [docs/architecture/diagrams/](#) remain the diagram source of truth. This document and its PDF are presentation views of the same implemented architecture.